

MECHATRONICS AND IOT

ASSIGNMENT - 3

SMART FLOOD LEVEL MONITORING SYSTEM USING ULTRASONIC SENSOR

TEAM MEMBERS:

JEEVA T	24MER020
NITHEESH M	24MER036
SANJAY S	24MER054
SIVA BALAJI S	24MER058
SIVA S	24MER059

SMART FLOOD LEVEL MONITORING SYSTEM USING

1. Project Overview

The Smart Flood Level Monitoring System is an IoT-based system designed to continuously monitor water levels and provide early warnings during critical flood conditions. Flooding can cause significant damage to infrastructure, agriculture, property, and human life. Early detection of rising water levels can help authorities and communities take preventive action.

The proposed system uses an ultrasonic sensor to measure the distance between the sensor and the surface of the water. An ESP8266 NodeMCU processes the measured distance and determines the corresponding water-level condition.

The system provides local alerts using LEDs and a buzzer and can transmit water-level information to a cloud platform such as Adafruit IO through Wi-Fi. This enables remote monitoring of water levels and supports disaster-management applications.

Main components

- GPS NEO-6M
- ESP32
- OLED
- Buzzer
- MQ2
- Pulse sensor
- DHT-11

2. Project Statement

Floods are one of the major natural hazards that can affect human settlements and infrastructure. Conventional flood monitoring methods may require manual observation and may not provide continuous real-time information.

A smart IoT-based flood monitoring system is proposed to continuously measure water levels using an ultrasonic sensor. The system analyzes the measured water level and generates warning alerts when the water reaches critical levels. The information can also be transmitted to a cloud platform for remote monitoring and data analysis.

3. Proposed Solution

The proposed system uses an ultrasonic sensor mounted above a water source to measure the distance from the sensor to the water surface.

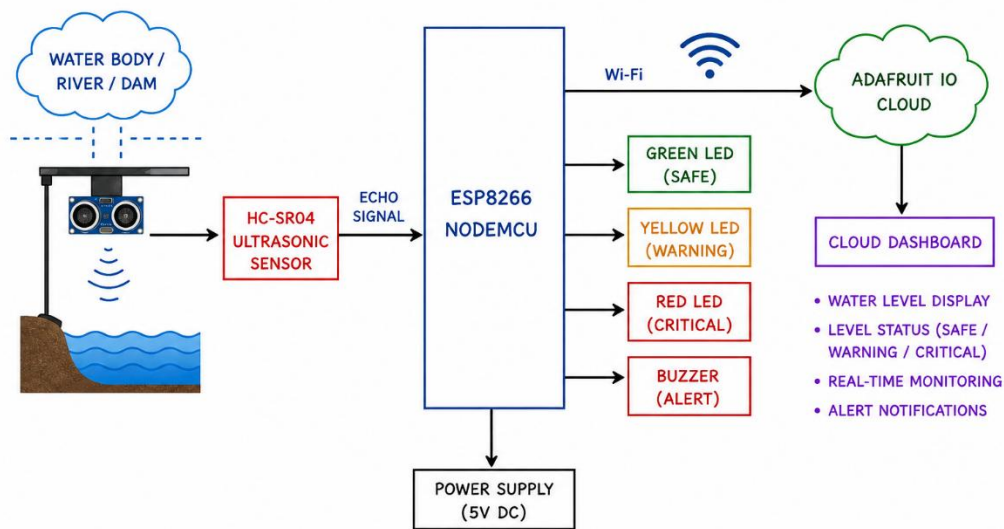
The major components are:

- **HC-SR04 Ultrasonic Sensor** – measures the distance to the water surface.
- **ESP8266 NodeMCU** – processes the sensor readings and provides Wi-Fi connectivity.
- **Green LED** – indicates a safe water level.
- **Yellow LED** – indicates a warning condition.
- **Red LED** – indicates a critical flood condition.
- **Buzzer** – provides an audible warning during critical conditions.
- **Adafruit IO** – stores and displays water-level data remotely.

As the water level rises, the distance measured by the ultrasonic sensor decreases. The ESP8266 uses this distance to determine whether the water level is safe, warning, or critical.

4. System Architecture:

SYSTEM ARCHITECTURE



5. Circuit Table:

CIRCUIT TABLE

1. MQ135 GAS SENSOR CONNECTION

Component	Pin	ESP8266 (NodeMCU) Pin	Function
MQ135 Gas Sensor	VCC	VIN / 5V	Power Supply (5V)
	GND	GND	Ground
	AO	A0	Analog Output to ESP8266

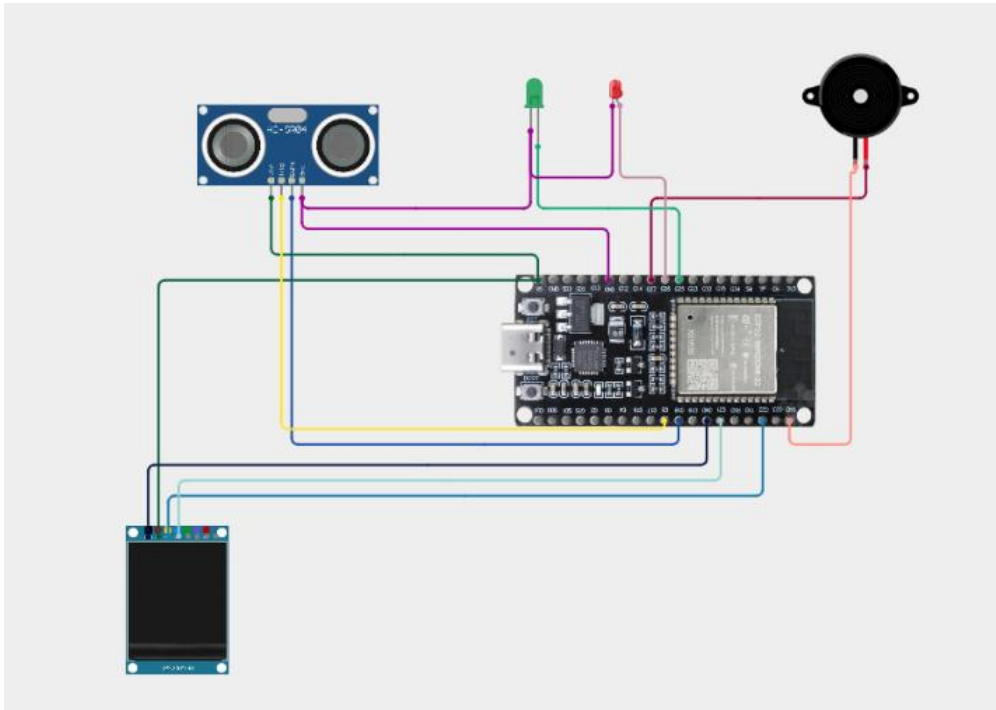
2. LED CONNECTIONS

Component	LED Pin	ESP8266 (NodeMCU) Pin	Other Connection	Function
Green LED	Anode (+)	D5 (GPIO14)	Through 220 Ω Resistor	Indicates Good Air Quality
	Cathode (-)	GND	Direct to GND	–
Red LED	Anode (+)	D6 (GPIO12)	Through 220 Ω Resistor	Indicates Poor Air Quality
	Cathode (-)	GND	Direct to GND	–

3. POWER SUPPLY

Component	Pin	Connection	Function
ESP8266 NodeMCU	VIN	5V DC (USB / External 5V Supply)	Power Supply to ESP8266
ESP8266 NodeMCU	GND	Common Ground	Ground

6. Circuit Design:



7. Algorithm:

Step 1: Start the system.

Step 2: Initialize the ESP32 and configure the input and output pins.

Step 3: Initialize the OLED display.

Step 4: Initialize the MQ-2 gas sensor, pulse sensor, and DHT-11 sensor.

Step 5: Initialize the NEO-6M GPS module and wait for valid GPS data.

Step 6: Read the temperature and humidity values from the DHT-11 sensor.

Step 7: Read the gas/smoke level from the MQ-2 sensor.

Step 8: Read the pulse signal from the pulse sensor and calculate the pulse rate.

Step 9: Read the latitude and longitude information from the GPS module when a valid satellite fix is available.

Step 10: Process and compare the sensor readings with the predefined threshold values.

Step 11: If the gas level is within the safe range, continue normal operation. If it exceeds the defined threshold, classify the condition as gas/smoke detected and activate the buzzer.

Step 12: Check the pulse reading. If it is outside the selected monitoring range, indicate an abnormal pulse condition.

Step 13: Display the temperature, humidity, gas level, pulse rate, and GPS information on the OLED display.

Step 14: If any critical condition is detected, activate the buzzer and display an appropriate warning message.

Step 15: Repeat the sensing and monitoring process continuously.

8. Coding:

```
#include <WiFi.h>

#include "AdafruitIO_WiFi.h"

#include <Wire.h>

#include <LiquidCrystal_I2C.h>

// =====
// WIFI DETAILS
// =====

#define WIFI_SSID   "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

// =====
// ADAFRUIT IO DETAILS
// =====

#define IO_USERNAME "YOUR_ADAFRUIT_USERNAME"
#define IO_KEY      "YOUR_ADAFRUIT_IO_KEY"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASSWORD);

// =====
// ADAFRUIT IO FEEDS
```

```

// =====
AdafruitIO_Feed *waterDistanceFeed = io.feed("water-distance");
AdafruitIO_Feed *waterLevelFeed  = io.feed("water-level");
AdafruitIO_Feed *floodStatusFeed  = io.feed("flood-status");
// =====
// PIN CONNECTIONS
// =====
// Ultrasonic sensor
#define TRIG_PIN 5
#define ECHO_PIN 18
// LEDs
#define GREEN_LED 25
#define RED_LED 26
// Buzzer
#define BUZZER 27
// LCD
#define SDA_PIN 21
#define SCL_PIN 22
LiquidCrystal_I2C lcd(0x27, 16, 2);
// =====
// FLOOD LEVEL SETTINGS
// =====
// Distance between ultrasonic sensor and water
#define SAFE_DISTANCE  30
#define DANGER_DISTANCE 10
// =====
// SETUP
// =====
void setup()

```

```
{
  Serial.begin(115200);
  // Ultrasonic
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  // LEDs
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  // Buzzer
  pinMode(BUZZER, OUTPUT);
  // LCD
  Wire.begin(SDA_PIN, SCL_PIN);
  lcd.init();
  lcd.backlight();
  // Startup message
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SMART FLOOD");
  lcd.setCursor(0, 1);
  lcd.print("MONITORING");
  delay(2000);
  lcd.clear();
  // =====
  // CONNECT TO ADAFRUIT IO
  // =====
  Serial.println();
  Serial.println("Connecting to Adafruit IO...");
  io.connect();
  while (io.status() < AIO_CONNECTED)
```



```
{  
  Serial.print(".");  
  delay(500);  
}  
  
Serial.println();  
Serial.println("Adafruit IO Connected!");  
  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("Adafruit IO");  
lcd.setCursor(0, 1);  
lcd.print("Connected!");  
delay(2000);  
lcd.clear();  
}  
  
// =====  
// GET ULTRASONIC DISTANCE  
// =====  
  
float getDistance()  
{  
  digitalWrite(TRIG_PIN, LOW);  
  delayMicroseconds(2);  
  digitalWrite(TRIG_PIN, HIGH);  
  delayMicroseconds(10);  
  digitalWrite(TRIG_PIN, LOW);  
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);  
  if (duration == 0)  
  {  
    return -1;  
  }  
}
```

```

float distance = duration * 0.0343 / 2;

return distance;

}

// =====

// CALCULATE WATER LEVEL

// =====

int calculateWaterLevel(float distance)
{
    /*
        Maximum safe measurement = 30 cm
        Minimum measurement = 0 cm
        As distance decreases,
        water level percentage increases.
    */
    int level = ((SAFE_DISTANCE - distance) / SAFE_DISTANCE) * 100;
    if (level < 0)
        level = 0;
    if (level > 100)
        level = 100;
    return level;
}

// =====

// MAIN LOOP

// =====

void loop()
{
    // Keep Adafruit IO connection alive
    io.run();

    // Read distance

```

```
float distance = getDistance();

// Sensor error
if (distance < 0)
{
    Serial.println("Ultrasonic sensor error!");
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Sensor Error!");
    lcd.setCursor(0, 1);
    lcd.print("Check HC-SR04");
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, LOW);
    delay(2000);
    return;
}

// Calculate water level
int waterLevel = calculateWaterLevel(distance);

// Variables for status
String floodStatus;

// =====
// SAFE CONDITION
// =====

if (distance > SAFE_DISTANCE)
{
    floodStatus = "SAFE";
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, LOW);
}
```

```

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("Water:");

lcd.print(distance, 1);

lcd.print("cm");

lcd.setCursor(0, 1);

lcd.print("STATUS: SAFE");

}

// =====

// WARNING CONDITION

// =====

else if (distance > DANGER_DISTANCE &&
        distance <= SAFE_DISTANCE)
{
    floodStatus = "WARNING";

    digitalWrite(GREEN_LED, LOW);

    digitalWrite(RED_LED, HIGH);

    // Beeping buzzer

    digitalWrite(BUZZER, HIGH);

    delay(200);

    digitalWrite(BUZZER, LOW);

    lcd.clear();

    lcd.setCursor(0, 0);

    lcd.print("Water:");

    lcd.print(distance, 1);

    lcd.print("cm");

    lcd.setCursor(0, 1);

    lcd.print("WARNING!");

}

```

```

// =====
// FLOOD CONDITION
// =====
else
{
    floodStatus = "FLOOD";
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    // Continuous buzzer
    digitalWrite(BUZZER, HIGH);
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Water:");
    lcd.print(distance, 1);
    lcd.print("cm");
    lcd.setCursor(0, 1);
    lcd.print("!!! FLOOD !!!");
}
// =====
// SERIAL MONITOR
// =====
Serial.println("-----");
Serial.print("Distance: ");
Serial.print(distance);
Serial.println(" cm");
Serial.print("Water Level: ");
Serial.print(waterLevel);
Serial.println(" %");
Serial.print("Status: ");

```

```

Serial.println(floodStatus);

// =====

// SEND DATA TO ADAFRUIT IO

// =====

waterDistanceFeed->save(distance);

waterLevelFeed->save(waterLevel);

floodStatusFeed->save(floodStatus.c_str());

Serial.println("Data sent to Adafruit IO!");

Serial.println("-----");

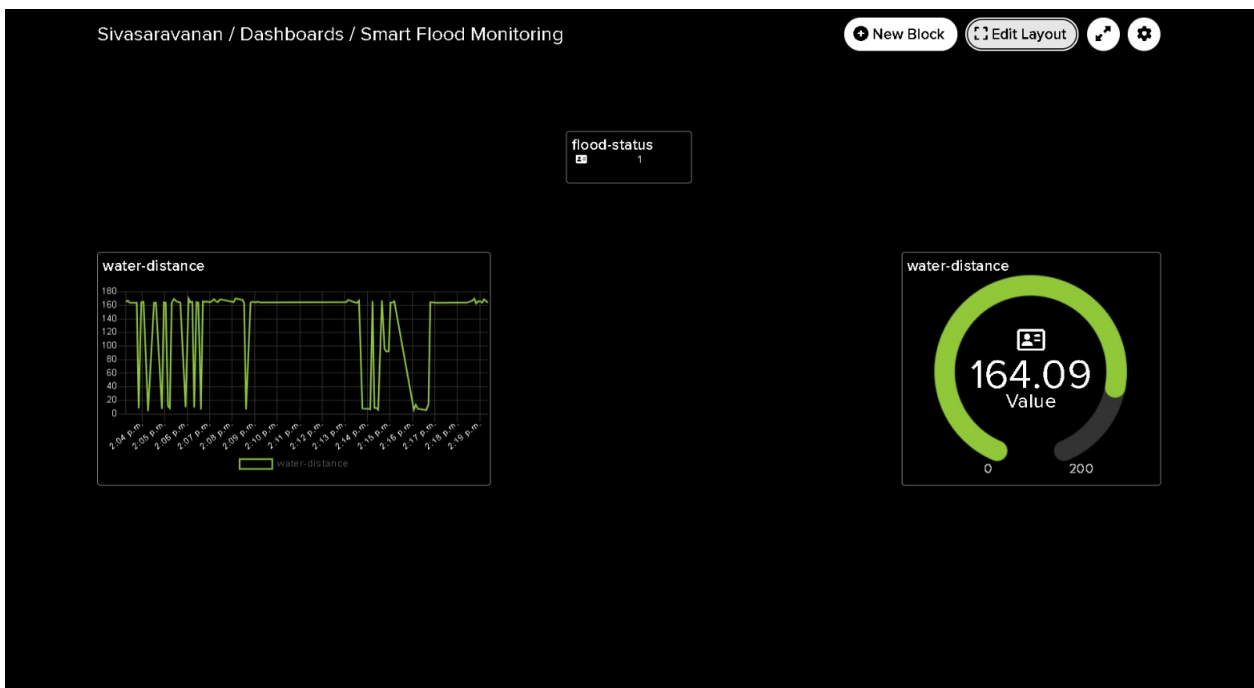
// Wait before next reading

delay(5000);

}

```

9. System Working :



The proposed system is an ESP32-based multi-parameter monitoring and alert system that integrates the NEO-6M GPS, OLED display, MQ-2 gas sensor, pulse sensor, and DHT-11 temperature and humidity sensor. The ESP32 acts as the central controller and continuously collects data from all connected sensors.

When the system is powered ON, the ESP32 initializes all sensors and the OLED display. The DHT-11 sensor measures the surrounding temperature and humidity, while the MQ-2 sensor detects the presence of smoke and combustible gases. The pulse sensor detects the user's pulse signal and provides the pulse-rate reading. At the same time, the NEO-6M GPS module receives signals from satellites and provides the geographical location in terms of latitude and longitude when a valid GPS fix is available.

The ESP32 processes the sensor readings and displays the relevant information on the OLED display. The display can be used to show parameters such as temperature, humidity, gas level, pulse rate, and GPS location.

The system also performs basic threshold-based monitoring. If the MQ-2 gas reading exceeds the predefined threshold, the ESP32 identifies a potentially unsafe gas or smoke condition and activates the buzzer to provide an audible warning. Similarly, if the pulse reading falls outside the selected monitoring range, the system can display an appropriate warning.

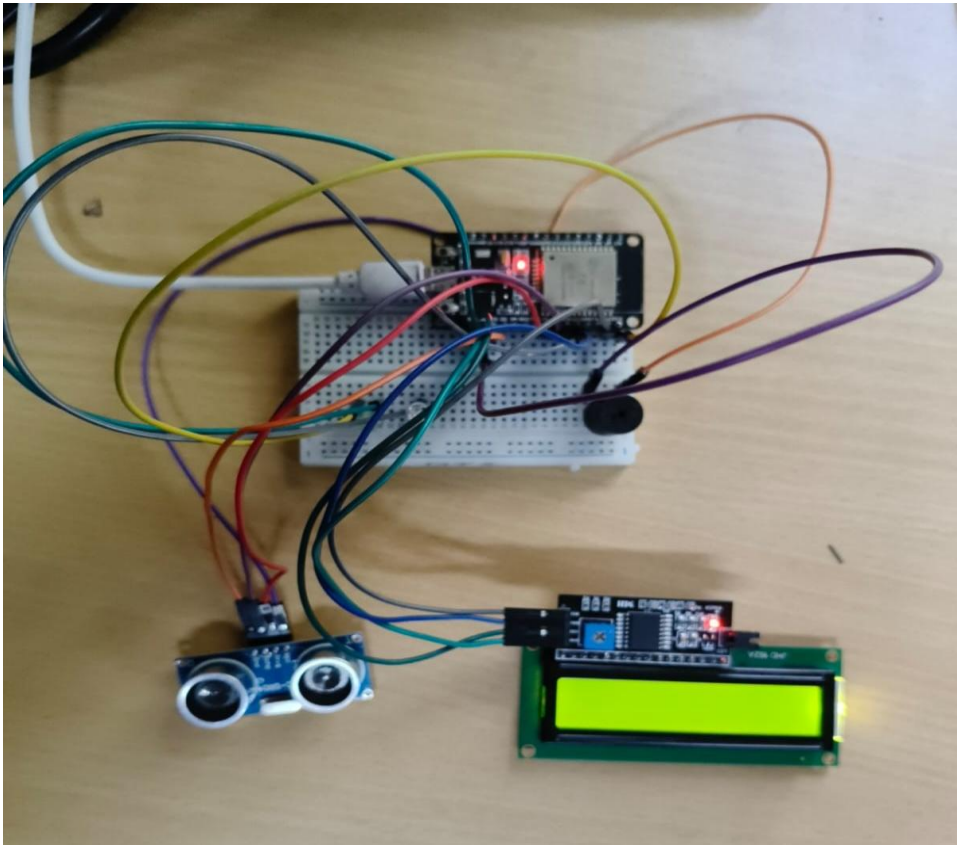
The GPS module provides the user's current location, which can be useful for identifying the location associated with an alert. The system continuously updates the sensor readings and repeats the monitoring process.

10. BILL OF MATERIAL:

BILL OF MATERIAL

S.No.	Component	Specification / Description	Quantity
1.	 GPS NEO-6M	GPS Positioning Module (NEO-6M) with Antenna	1
2.	 ESP32	ESP32 Wi-Fi + Bluetooth Development Board	1
3.	 OLED	0.96-inch I2C OLED Display Module	1
4.	 Buzzer	Buzzer for Audible Alerts (3V – 5V)	1
5.	 MQ2	MQ-2 Gas / Smoke Sensor Module	1
6.	 Pulse Sensor	Pulse / Heart Rate Sensor Module	1
7.	 DHT-11	Temperature and Humidity Sensor	1

11. Prototype Implementation:



The prototype is implemented using an ESP8266 NodeMCU connected to an HC-SR04 ultrasonic sensor. The ultrasonic sensor is positioned above a water container to measure the water surface level.

Three LEDs are used to indicate different water-level conditions. The green LED indicates a safe water level, the yellow LED indicates a warning condition, and the red LED indicates a critical flood condition. A buzzer is activated during the critical condition.

The ESP8266 is connected to Wi-Fi and can transmit the measured water-level information to the Adafruit IO cloud platform. The cloud dashboard can display the current water level and historical readings.

12. Results and Performance Analysis:

The developed system was successfully implemented using ESP32, NEO-6M GPS, OLED display, buzzer, MQ-2 gas sensor, pulse sensor, and DHT-11 temperature and humidity sensor. The prototype was tested to verify the sensing, processing, display, location tracking, and alert functions of the system.

The DHT-11 sensor successfully measured the surrounding temperature and humidity, while the MQ-2 sensor detected changes in smoke and combustible-gas levels. The pulse sensor was used to obtain the user's pulse rate. The NEO-6M GPS module provided geographical location information, which was processed by the ESP32.

The collected sensor information was displayed on the OLED display, allowing the user to view important parameters locally. When an abnormal gas level or other predefined critical condition was detected, the buzzer was activated to provide an audible warning.

Test Results

- During normal operating conditions, the system displayed the measured temperature, humidity, gas level, and pulse rate on the OLED display. When the MQ-2 sensor detected an increase in smoke

or gas concentration, the system identified the condition as abnormal and activated the buzzer.

- The GPS module provided latitude and longitude information after obtaining a sufficient satellite fix. GPS acquisition time and accuracy may vary depending on the antenna position, surrounding buildings, and availability of satellites.
- The DHT-11 provided environmental temperature and humidity readings, while the pulse sensor continuously detected pulse signals. The ESP32 successfully collected and processed data from all connected sensors.

Overall Performance

- The system demonstrated effective multi-parameter monitoring by combining environmental, gas, physiological, and location information in a single ESP32-based platform. The OLED display provided immediate local access to the sensor readings, while the buzzer provided a quick warning during critical conditions.
- The prototype is suitable for demonstration and basic monitoring applications. However, the MQ-2 and pulse sensors require proper calibration, and the DHT-11 has limited accuracy compared with higher-grade sensors. Similarly, GPS accuracy depends on satellite availability and environmental conditions. Therefore, for medical, industrial, or safety-critical applications, calibrated and certified sensors should be used.